

AUTONATEAI · SEASON 1



COURSE WORKBOOK

THE CONTEXT WINDOW

Prompting, Context, Memory & Graphs — how to think with AI

A six-episode animated CS series. Five people at a tiny AI studio have thirty days until Demo Day — and every episode, the way they work with AI breaks in a new, very human way. The Operator, the voice in everybody's head, takes us inside the machine (and the brain) to show why, then teaches the fix with real prompts, real code, and real SQL.

How this course works

A six-episode animated CS series. Five people at a tiny AI studio have thirty days until Demo Day — and every episode, the way they work with AI breaks in a new, very human way. The Operator, the voice in everybody's head, takes us inside the machine (and the brain) to show why, then teaches the fix with real prompts, real code, and real SQL.

Each episode is 3:31–4:54 long (26 minutes for the season). Watch it, then work the matching pages here: the big idea, the science underneath it, the terms, one exercise you do on a real problem, a template you keep, and the questions to ask before anything ships.

THE SEASON



EPISODE 1 · 4:54
Garbage In



EPISODE 2 · 4:21
The Room, Not Just the Ask



EPISODE 3 · 3:31
Role, Task, Constraints, Format



EPISODE 4 · 4:18
That's Not a Source



EPISODE 5 · 4:31
The Missing Ledger



EPISODE 6 · 4:32
The Web Under the Table

Watch the season: <https://www.youtube.com/playlist?list=PLVomZxd75ZEs>

THE CAST

Dre — Co-founder · types first, thinks later

Brent — Head of Growth · has a vest · learns everything the hard way

Arjun — Data engineer · the database whisperer

Nia — Co-founder · ten years writing city grant scopes · wants receipts

Mei — ML engineer · has read the paper

The Operator — Narrator · lives in everybody's head · goes inside the machine



EPISODE 1

Garbage In

4:54 · Context Engineering: From Prompts to Knowledge Graphs

A vague prompt gets the most **average** answer. A trusted answer gets you a confident mistake.

WHAT YOU'LL LEARN

- What a language model actually does: predict the next token, over and over
- The two opposite failures: asking too little, trusting too much
- How to name what went wrong instead of blaming the tool

KEY TERMS

Next-token prediction

The model's one job: given everything so far, produce the most likely next piece of text.

Specificity

Which file, which problem, what not to touch, what done looks like.

Ground truth

The source you check an answer against — the file, the data, the docs, running the code.

INSIDE THE MACHINE

Language models are networks of artificial neurons that learn by adjusting connection strengths — the bet Geoffrey Hinton and colleagues made for decades. Vague words give the model a starting point, not instructions, and it fills the gap with the most typical continuation.

TRY THIS

The four-word autopsy

1. Find a prompt you sent an AI tool this week that disappointed you.
2. Rewrite it with: the exact file or input, the exact problem, one thing it must not touch, and what the finished output looks like.
3. Run both. Write two sentences on what changed — and what the first answer was 'averaging' toward.

KEEP THIS

```
In <file>, <function> breaks when <specific case>.
Fix only that. Don't touch <area>.
Show me <the changed function / a diff / a one-line
note>.
```

BEFORE IT SHIPS

- ☐ Did I say which part?
- ☐ Did I say what not to touch?
- ☐ Do I know exactly what it changed?
- ☐ Did I check the answer against the source?

Sources. Rumelhart, Hinton & Williams (1986). Learning representations by back-propagating errors. Nature.



EPISODE 2

The Room, Not Just the Ask

4:21 · Context Engineering: From Prompts to Knowledge Graphs

A model can only work with what's in its context window. Too little and it guesses; too much and it drowns.

WHAT YOU'LL LEARN

- What the context window is — and what fills it without you noticing
- Why flooding it backfires ('lost in the middle')
- Four context-engineering moves and a project context file

KEY TERMS

Context window

Everything the model can see for this request, measured in tokens. Its working memory.

Attention

The mechanism that lets every token weigh every other token when deciding what matters.

Lost in the middle

Models use information at the start and end of a long context better than information buried in the middle.

INSIDE THE MACHINE

Human working memory holds about four chunks at once (Cowan, 2001), and people recall the first and last items of a list best (the serial-position effect). Long-context language models show a strikingly similar shape of forgetting.

TRY THIS

Build the room

1. Pick a real bug or task. List every file/document you'd normally paste in.
2. Cross out everything the task doesn't touch. Summarize anything longer than a page into its decisions.
3. Start the session with: 'Before you change anything: what files can you see, and what are you assuming?'
4. Write a 6-line project context file (AGENTS.md / CLAUDE.md) with the rules you keep repeating.

KEEP THIS

```
# Project context
- Runtime + versions, and "no new dependencies without asking"
- Where the important logic lives (and what not to duplicate)
- House rules (never delete TODOs, tests before refactors)
- Files to ask before touching
```

BEFORE IT SHIPS

- ☐ Did I give it only what the task touches?
- ☐ Did I summarize instead of paste?
- ☐ Did I ask what it can see before it acted?
- ☐ Is the context written down once, not re-explained daily?

Sources. Cowan, N. (2001). The magical number 4 in short-term memory. Behavioral and Brain Sciences. · Murdock, B. (1962). The serial position effect of free recall. Journal of Experimental Psychology. · Liu, N. F. et al. (2023). Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172.



EPISODE 3

Role, Task, Constraints, Format

3:31 · Context Engineering: From Prompts to Knowledge Graphs

A prompt is a scope of work. Four lines turn a wish into a spec — and you decide what done looks like before you start.

WHAT YOU'LL LEARN

- The four-row prompt: Role, Task, Constraints, Format (+ Done)
- Why a role works like priming
- Treat the first answer as a draft, not a verdict

KEY TERMS

Role

A lens, not a compliment: who is speaking, to whom.

Constraints

The fence: length, tone, what to avoid, what must be included.

Definition of done

Checkable criteria written before the work starts.

INSIDE THE MACHINE

Semantic priming: seeing 'doctor' makes 'nurse' faster to recognize (Meyer & Schvaneveldt, 1971). A role in a prompt tilts which of the millions of voices a model learned from shapes the answer.

TRY THIS

Scope it like a contract

1. Take a real message you need to write (a follow-up, a bug report, a cover letter).
2. Write the four rows. Then add DONE: three checkable criteria.
3. Generate, then do one revision pass that cuts something and adds one specific detail.
4. Check the result against DONE. If it fails, fix the prompt, not just the output.

KEEP THIS

```
ROLE: <who is speaking, to whom>
TASK: <one job, named>
CONSTRAINTS: <length · tone · must include · must avoid>
FORMAT: <the shape of the answer>
DONE: <3 checkable criteria>
```

BEFORE IT SHIPS

- ☐ Is the role a lens or a compliment?
- ☐ Is there exactly one task?
- ☐ Could someone else check DONE without asking me?

Sources. Meyer, D. E. & Schvaneveldt, R. W. (1971). Facilitation in recognizing pairs of words. Journal of Experimental Psychology.



EPISODE 4

That's Not a Source

4:18 · Context Engineering: From Prompts to Knowledge Graphs

Fluent isn't factual. A sentence isn't a source — check what would hurt if it were wrong.

WHAT YOU'LL LEARN

- Why models produce plausible fabrications (and why people do too)
- The ten-second verification kit
- Package hallucinations and slopsquatting — the security risk
- Closing ambiguity before the model closes it for you

KEY TERMS

Confabulation

Producing a confident, plausible account that isn't grounded in fact — Hinton's preferred word over 'hallucination'.

Slopsquatting

Registering package names that AI models tend to invent, so the next person who installs one gets the attacker's code.

Ambiguity

Any decision the prompt left open (which key? keep first or last?) that the model will silently decide.

INSIDE THE MACHINE

Every eye has a blind spot where the optic nerve leaves the retina; the brain fills it in with a confident guess. Models are trained to be plausible, not true. The difference is that you can check.

TRY THIS

Catch one in the wild

1. Ask an AI tool for code that uses a library you've never used.
2. Before running anything: `npm view <package>` (or `pip index / docs`) for every import, and check every method it calls exists.
3. Rewrite the prompt to close two ambiguities (e.g., dedupe key, keep first vs last) and compare.

KEEP THIS

```
npm view <package> name          # does it exist?
node -p "typeof [].someMethod"    # does the method exist?
# read the official docs · run it on a tiny real input
```

BEFORE IT SHIPS

- ☐ Does every package and method actually exist?
- ☐ Did I close the ambiguities that matter?
- ☐ Am I verifying what would hurt — not arithmetic?

Sources. Spracklen, J. et al. (2025). We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. USENIX Security.



EPISODE 5

The Missing Ledger

4:31 · Context Engineering: From Prompts to Knowledge Graphs

Memory is a storyteller; a database is a witness. Model the truth once, then ask it precise questions.

WHAT YOU'LL LEARN

- File vs. in-memory object vs. database — three different deals
- Tables, primary keys, foreign keys: one row per truth
- SELECT, WHERE, JOIN — and the JOIN that silently inflates COUNT
- Querying from code so the answer is the same every time

KEY TERMS

Primary key

The column that uniquely identifies a row (each person exists once).

Foreign key

A column that points at another table's primary key instead of copying its data.

Fan-out

A JOIN that produces one row per matching child, so COUNT(*) counts children, not parents. Use COUNT(DISTINCT ...).

INSIDE THE MACHINE

Human memory is reconstructive: in Loftus & Palmer (1974), changing 'hit' to 'smashed' in a question made people more likely to 'remember' broken glass that never existed. Models don't remember your data between sessions at all. A system of record is what remembers exactly.

TRY THIS

Build the ledger

1. Create people, ideas and feedback tables in SQLite (feedback points at both with foreign keys).
2. Insert 5 people (one duplicate under two spellings), 3 ideas, 8 feedback rows (one person comments 3 times).
3. Count 'people who gave feedback on idea 3' with COUNT(*) after a JOIN, then with COUNT(DISTINCT person_id). Explain the difference in one sentence.

KEEP THIS

```
SELECT COUNT(DISTINCT person_id) AS people
FROM feedback
WHERE idea_id = 3;
```

BEFORE IT SHIPS

- ☐ Does each real-world thing exist exactly once?
- ☐ Am I counting rows or the things I actually mean?
- ☐ Is the question answered by code, not by memory?

Sources. Loftus, E. F. & Palmer, J. C. (1974). Reconstruction of automobile destruction. Journal of Verbal Learning and Verbal Behavior. · Bartlett, F. C. (1932). Remembering. Cambridge University Press.



EPISODE 6

The Web Under the Table

4:32 · Context Engineering: From Prompts to Knowledge Graphs

When relationships repeat, loop, and carry their own facts, they're a graph. The connections are the knowledge — for brains, neural nets, and your AI's context.

WHAT YOU'LL LEARN

- The tell that a table should be a graph
- Nodes, directed edges, hops, cycles — and traversal with WITH RECURSIVE
- Feeding a graph path to an AI as context, with sources

KEY TERMS

Node / edge

A thing / a connection between two things. Edges can carry their own data (where, when).

Traversal

Walking a graph hop by hop from a starting node.

Retrieval as context

Giving the model the few facts (the path, the rows) a question needs — not the whole database.

INSIDE THE MACHINE

A brain is a graph: ~86 billion neurons, something like 100 trillion synapses. 'Cells that fire together wire together' (Schatz, summarizing Hebb). In neural networks too, the knowledge lives in the connection strengths.

TRY THIS

Shortest way in

1. Add an introductions table (from_name, to_name, venue, date) with at least one person introduced twice and one cycle.
2. Write the WITH RECURSIVE query that finds the shortest path from you to a target, without walking in circles.
3. Give an AI only that path plus the source rows and ask it to draft an intro request. Compare with giving it the whole table.

KEEP THIS

```
WITH RECURSIVE path(person, hops, route) AS (  
  SELECT 'Nia', 0, 'Nia'  
  UNION ALL  
  SELECT i.to_name, p.hops + 1, p.route || ' → ' ||  
i.to_name  
  FROM introductions i JOIN path p ON i.from_name =  
p.person  
  WHERE p.hops < 4 AND instr(p.route, i.to_name) = 0  
)  
SELECT route FROM path WHERE person = 'Fairview Parks'  
ORDER BY hops LIMIT 1;
```

BEFORE IT SHIPS

- Does this relationship have facts of its own? → its own table
- Can my traversal loop forever? → stop on revisits
- Am I giving the AI the path, or the haystack?

Sources. Hebb, D. O. (1949). The Organization of Behavior. Wiley. · Azevedo et al. (2009), J. Comp. Neurology — ~86 billion neurons.

The season on one page

EPISODE 1

Garbage In

A vague prompt gets the most **average** answer. A trusted answer gets you a confident mistake.

EPISODE 2

The Room, Not Just the Ask

A model can only work with what's in its context window. Too little and it guesses; too much and it drowns.

EPISODE 3

Role, Task, Constraints, Format

A prompt is a scope of work. Four lines turn a wish into a spec — and you decide what done looks like before you start.

EPISODE 4

That's Not a Source

Fluent isn't factual. A sentence isn't a source — check what would hurt if it were wrong.

EPISODE 5

The Missing Ledger

Memory is a storyteller; a database is a witness. Model the truth once, then ask it precise questions.

EPISODE 6

The Web Under the Table

When relationships repeat, loop, and carry their own facts, they're a graph. The connections are the knowledge — for brains, neural nets, and your AI's context.

THE FOUR-ROW PROMPT

```
ROLE: <who is speaking, to whom>
TASK: <one job, named>
CONSTRAINTS: <length · tone · must include ·
must avoid>
FORMAT: <the shape of the answer>
DONE: <3 checkable criteria>
```

THE TEN-SECOND VERIFICATION KIT

```
npm view <package> name          # does it
exist?
node -p "typeof [].someMethod" # does the
method exist?
# read the official docs · run it on a tiny
real input
```

KEEP GOING

The full written lessons — Prompt & Context Engineering and Relational Databases & Graphs — and new seasons live at autonateai.com. Questions or want this for your class or team? nate@autonateai.com